



University of
Nottingham
UK | CHINA | MALAYSIA

Lecture 9: Unsupervised Learning and KNN Classification

COMP1037 Fundamentals of Artificial Intelligence

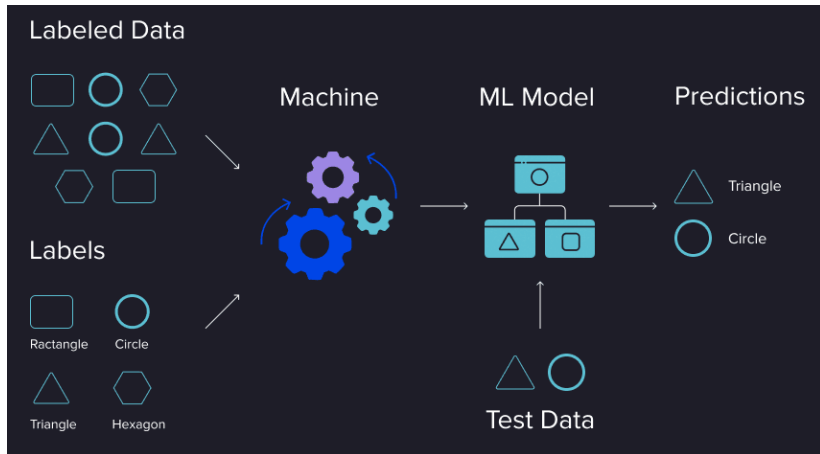
Xinan Chen

University of Nottingham Ningbo China

April 15, 2025

Outline

1. Introduction to Unsupervised Learning
2. Clustering
3. KNN
4. Dimensionality Reduction



What is Supervised Learning?

Definition

A type of machine learning where the model learns from labeled data to predict outcomes.

Key Concepts:

- Explicit supervision with labeled data (input-output pairs).
- The algorithm learns a mapping from inputs to outputs.
- Commonly used for classification and regression tasks.



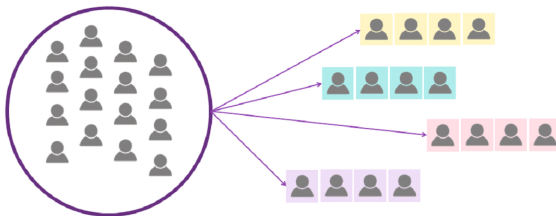
What is Unsupervised Learning?

Definition

A type of machine learning where the model learns patterns from unlabeled data.

Key Concepts:

- No explicit supervision.
- The algorithm identifies hidden structures or patterns.
- Commonly used for **clustering** and **dimensionality reduction**.



Supervised vs. Unsupervised Learning

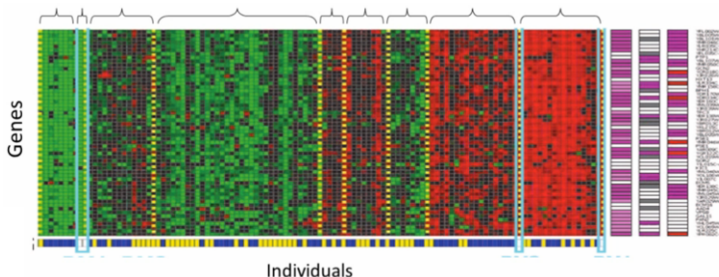
Aspect	Supervised Learning	Unsupervised Learning
Data	Labeled	Unlabeled
Goal	Predict outcomes	Discover patterns
Examples	Classification, Regression	Clustering, Dimension Reduction

Table

Comparison of Supervised and Unsupervised Learning

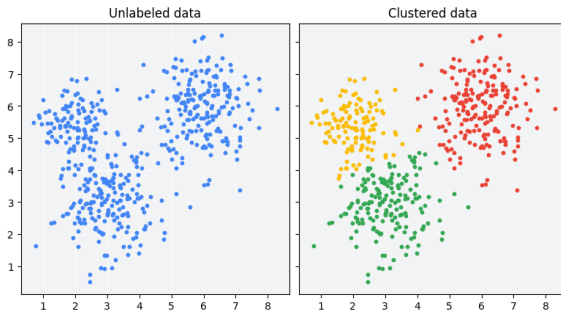
Applications of Unsupervised Learning

- **Customer Segmentation:** Grouping customers based on purchasing behavior.
- **Anomaly Detection:** Identifying unusual patterns in data (e.g., fraud detection).
- **Image Compression:** Reducing the size of images using clustering.
- **Recommendation Systems:** Suggesting products based on user behavior.



Applications of Unsupervised Learning (cont.)

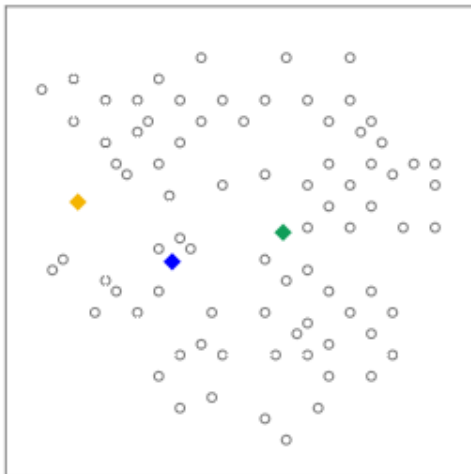
- **Google News:** Groups news stories into cohesive groups.
- **Genomics Microarray Data:** Cluster individual's genes into types of people.
- **Organize Computer Clusters:** Identify potential weak spots or distribute workload effectively.
- **Social Network Analysis:** Uncover valuable insights from the structure and dynamics of social networks.



Types of Unsupervised Learning

- **Clustering:** Grouping similar data points (e.g., K-Means, Hierarchical Clustering).
- **Dimensionality Reduction:** Reducing the number of features (e.g., PCA, t-SNE).
- **Association Rule Learning:** Discovering relationships between variables (e.g., Apriori Algorithm).





k-means at initialization

What is Clustering?

Definition

Grouping similar data points together based on their features.

Intuition:

- Unsupervised learning technique.
- No predefined labels; the algorithm identifies natural groupings.
- Used for exploratory data analysis and pattern discovery.

Types of Clustering

- **Partitional Clustering:** Divides data into non-overlapping subsets (e.g., K-Means).
- **Hierarchical Clustering:** Builds a tree of clusters (e.g., Agglomerative, Divisive).
- **Density-Based Clustering:** Groups data based on density (e.g., DBSCAN).
- **Model-Based Clustering:** Assumes data is generated from a mixture of distributions (e.g., Gaussian Mixture Models).

K-Means Clustering

Definition

A type of unsupervised learning used for unlabeled data.

Goal: Find groups in the data, with the number of groups represented by K .

Algorithm:

- Works iteratively to assign each data point to one of K groups.
- Clusters data points based on feature similarity.

Results:

- **Centroids** of the K clusters (used to label new data).
- Labels for the training data (each point assigned to a single cluster).

K-means Clustering: Problem and Objective

Given a dataset $X = \{x_1, x_2, \dots, x_n\}$ where each $x_i \in \mathbb{R}^d$, the goal of K-means is to partition the dataset into k clusters $C = \{C_1, C_2, \dots, C_k\}$ such that the within-cluster sum of squares is minimized.

Objective Function:

$$J(C, \mu) = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

where:

- μ_i is the centroid of cluster C_i .
- $\|x - \mu_i\|^2$ is the squared Euclidean distance between a data point x and the centroid μ_i .

K-means Clustering: Algorithm Steps

The K-means algorithm proceeds as follows:

- 1. Initialization:** Randomly initialize k cluster centroids $\mu_1, \mu_2, \dots, \mu_k$.
- 2. Assignment Step:** Assign each data point x_j to the nearest centroid:

$$C_i = \{x_j : \|x_j - \mu_i\|^2 \leq \|x_j - \mu_l\|^2 \forall l \neq i\}$$

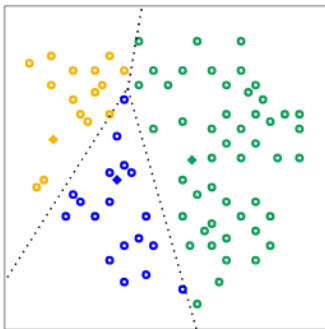
- 3. Update Step:** Recompute the centroids as the mean of all points assigned to each cluster:

$$\mu_i = \frac{1}{|C_i|} \sum_{x \in C_i} x$$

- 4. Repeat:** Repeat the assignment and update steps until convergence (i.e., when the centroids no longer change significantly or a maximum number of iterations is reached).

K-Means Algorithm - Example

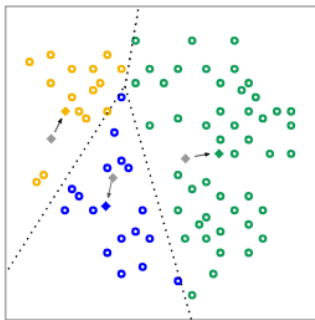
1. Provide an initial guess for K . For this example, we choose $K = 3$.
2. Randomly choose K centroids.



Initial clusters

K-Means Algorithm - Example (cont.)

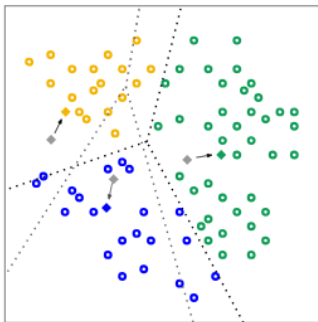
3. Assign each point to the nearest centroid to get K initial clusters.



Recomputed centroids

K-Means Algorithm - Example (cont.)

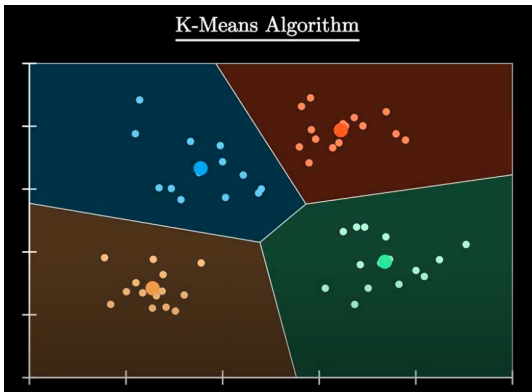
- For each cluster, calculate a new centroid by taking the mean position of all points in the cluster. The arrows in the figure show the change in centroid positions.



Clusters after reassignment.

K-Means Algorithm - Example (cont.)

5. Reassign each point to the nearest new centroid.
6. Repeat steps 4 and 5, recalculating centroids and cluster membership, until points no longer change clusters.



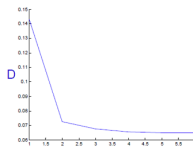
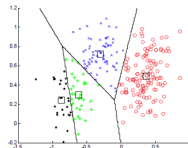
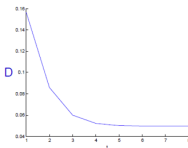
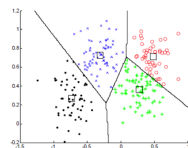
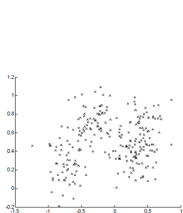
K-Means Algorithm Visualization

Video: K-Means Algorithm Visualization

https://www.youtube.com/watch?v=R2e3Ls9H_fc

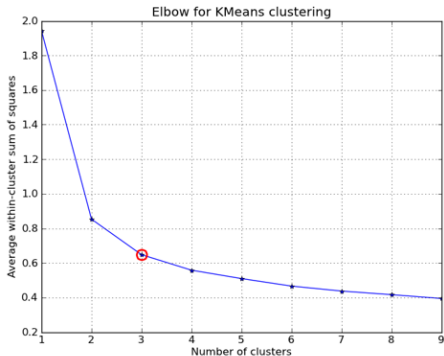
Play: K-Means Online

<https://www.naftaliharris.com/blog/visualizing-k-means-clustering/>



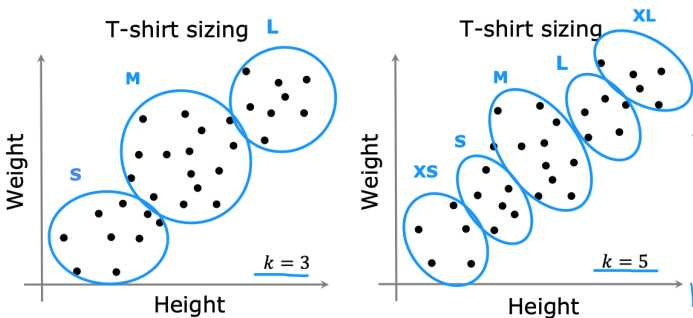
Limitations of K-Means

- Sensitive to **initial centroid** positions.
- Requires the number of clusters to be specified in advance.
- Struggles with non-spherical clusters and outliers.
- Not suitable for clusters of varying densities.



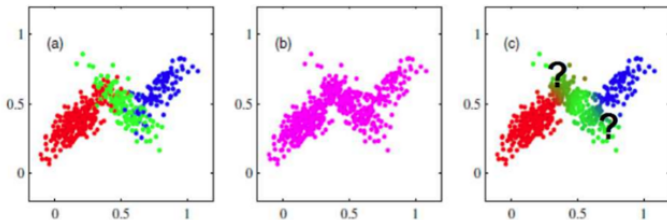
Choosing the Right Number of Clusters

- **Elbow Method:** Plot the sum of squared errors (SSE) vs. the number of clusters and look for the "elbow" point.
- **Silhouette Score:** Measures how similar a data point is to its own cluster compared to other clusters (range: -1 to 1).



Choosing the Right Number of Clusters (cont.)

Often, you want to get clusters for some later (downstream) purpose. Evaluate K-means based on how well it performs on that later purpose.

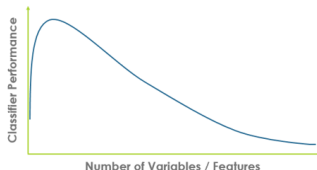


Hard vs Soft Assignments

- In K-means, there is a **hard assignment** of feature vectors to a cluster.
- However, for feature vectors near the boundary this may be a poor representation.
- Instead, we can consider a **soft-assignment**, where the strength of the assignment depends on distance.

To address these problems, we use soft clustering — **Gaussian Mixture Model**.

- Soft clustering is a form of clustering where observations may belong to multiple clusters.



What is K-Nearest Neighbors (KNN)?

Definition

KNN is a simple, non-parametric, **supervised** machine learning algorithm used for both classification and regression tasks.

Key Characteristics:

- **Instance-based (Lazy) Learning:** It does not build an explicit internal model during a training phase. Instead, it memorizes the training dataset.
- **Similarity Assumption:** It operates on the premise that similar things exist in close proximity to each other.
- **Supervised:** Unlike K-Means (which clusters unlabeled data), KNN requires labeled data to classify new observations.

The KNN Algorithm: Step-by-Step

To classify a new, unknown data point, the algorithm follows these steps:

1. **Choose K:** Select the number of neighbors, K .
2. **Calculate Distances:** Compute the distance between the new data point and every point in the training dataset.
3. **Find Neighbors:** Sort the distances in ascending order and select the top K nearest data points.
4. **Vote (Classification):** Count the class labels of these K neighbors.
5. **Assign Class:** Assign the new data point to the class that appears most frequently (majority vote).

Calculating Similarity: Distance Metrics

How do we define "nearest"? We use mathematical distance formulas. For continuous variables, the most common metrics are:

Euclidean Distance (L2 Norm)

The straight-line distance between two points p and q in n -dimensional space:

$$d(p, q) = \sqrt{\sum_{i=1}^n (q_i - p_i)^2}$$

Manhattan Distance (L1 Norm)

The distance measured along axes:

$$d(p, q) = \sum_{i=1}^n |q_i - p_i|$$

Choosing the Right K

Selecting K is crucial for model performance:

- **Small K (e.g., $K = 1$):** High variance, low bias. The model perfectly fits the training data but is highly sensitive to noise and outliers (Overfitting).
- **Large K :** Low variance, high bias. The decision boundaries become smoother, but it may ignore localized patterns (Underfitting).

Best Practices for K

- Choose an **odd number** for K to avoid tie votes in binary classification.
- A common rule of thumb is $K = \sqrt{N}$, where N is the number of data points.
- Use cross-validation to test different K values and select the one with the lowest error rate.

KNN Example: The Setup

Scenario: We want to classify a new ingredient into either "Fruit" or "Vegetable" based on two features: *Sweetness* (1-10) and *Crunchiness* (1-10).

Training Data:

Ingredient	Sweetness (x_1)	Crunchiness (x_2)	Class
Apple	8	7	Fruit
Banana	9	2	Fruit
Celery	2	9	Vegetable
Green Bean	3	6	Vegetable

New Observation (X): A Tomato with Sweetness = 6, Crunchiness = 4.

Is the Tomato a Fruit or a Vegetable? ($k=3$)

KNN Example: Calculating Distances

Let's calculate the Euclidean distance from the Tomato (6, 4) to our training points:

$$d = \sqrt{(x_{sweet} - 6)^2 + (x_{crunch} - 4)^2}$$

■ **Apple (8, 7):** $\sqrt{(8 - 6)^2 + (7 - 4)^2} = \sqrt{4 + 9} = \sqrt{13} \approx 3.6$

■ **Banana (9, 2):**
 $\sqrt{(9 - 6)^2 + (2 - 4)^2} = \sqrt{9 + 4} = \sqrt{13} \approx 3.6$

■ **Celery (2, 9):**
 $\sqrt{(2 - 6)^2 + (9 - 4)^2} = \sqrt{16 + 25} = \sqrt{41} \approx 6.4$

■ **Green Bean (3, 6):**
 $\sqrt{(3 - 6)^2 + (6 - 4)^2} = \sqrt{9 + 4} = \sqrt{13} \approx 3.6$

KNN Example: Voting and Prediction

Now, let's classify the Tomato using $K = 3$.

Step 1: Sort by Distance

1. Apple (Distance: 3.6) - **Fruit**
2. Banana (Distance: 3.6) - **Fruit**
3. Green Bean (Distance: 3.6) - **Vegetable**

Note: Celery is excluded because it is the furthest away (6.4).

Step 2: Majority Vote

- Fruits: 2 votes
- Vegetables: 1 vote

Result

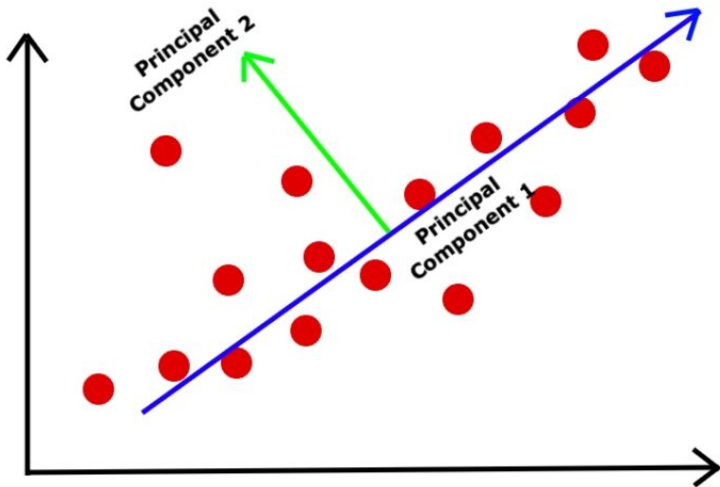
With a 2-to-1 majority, the KNN algorithm classifies the Tomato as a **Fruit**.

Pros and Cons of KNN

Advantages	Disadvantages
▶ Simple and easy to implement. ▶ No training period (lazy learner). ▶ Naturally handles multi-class problems. ▶ Highly adaptable to new data.	▶ Slow at testing phase (computes distance to all training points). ▶ Consumes a lot of memory. ▶ Sensitive to irrelevant features (Curse of Dimensionality). ▶ Requires feature scaling (Normalization/Standardization).

Comparing KNN and K-Means

Aspect	KNN	K-Means
Learning Type	Supervised Learning	Unsupervised Learning
Primary Goal	Predict the class or value of a new data point	Discover hidden groupings/clusters in data
Data Used	Requires labeled data	Uses unlabeled data
Meaning of K	Number of nearest neighbors to consider	Number of clusters to partition the data into
Training Phase	Lazy learner (No explicit training phase, just stores data)	Iterative optimization (Actively computes centroids)



Curse of Dimensionality

Explanation

As the number of dimensions increases, data becomes sparse, and distance metrics lose meaning.

Examples:

- High computational cost.
- Overfitting in machine learning models.
- Difficulty in visualizing and interpreting data.

What is Dimensionality Reduction?

Definition

A technique to reduce the number of features (dimensions) in a dataset while preserving its structure.

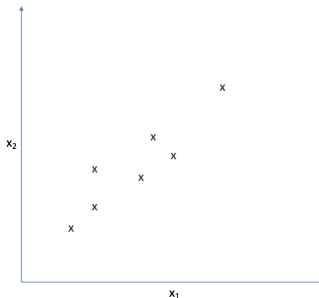
Motivation:

- Improves computational efficiency.
- Reduces noise and redundancy.
- Enables visualization of high-dimensional data.

Principal Component Analysis (PCA)

Theory

A linear dimensionality reduction technique that projects data onto orthogonal axes (principal components).



PCA Algorithm: Steps 1-2

Given a dataset $X = \{x_1, x_2, \dots, x_n\}$ where each $x_i \in \mathbb{R}^d$, the goal of PCA is to find a lower-dimensional representation of the data while preserving as much variance as possible.

1. Standardize the Data

Compute the mean of the dataset:

$$\bar{x} = \frac{1}{N} \sum_{i=1}^N x_i$$

Center the data:

$$X_{\text{centered}} = X - \bar{x}$$

2. Compute the Covariance Matrix

Covariance matrix captures the variance and correlation between features:

$$C = \frac{1}{N-1} X_{\text{centered}}^{\top} X_{\text{centered}}$$

PCA Algorithm: Steps 3–4

3. Compute the Eigenvalues and Eigenvectors

Solve the eigenvalue problem:

$$Cv_i = \lambda_i v_i$$

λ_i is the eigenvalue, v_i is the eigenvector.

4. Sort Eigenvectors by Eigenvalues

Order eigenvectors by decreasing eigenvalues:

$$\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_D$$

The eigenvectors corresponding to the largest eigenvalues capture the most variance.

PCA: Calculating Eigenvalues and Eigenvectors

1. Finding Eigenvalues (λ)

To find the eigenvalues, we solve the **characteristic equation**:

$$\det(C - \lambda I) = 0$$

where I is the identity matrix of the same dimension as C , and $\det(\cdot)$ denotes the determinant. Solving this polynomial equation yields the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_d$.

2. Finding Eigenvectors (v)

For each calculated eigenvalue λ_i , we find its corresponding eigenvector v_i by solving the homogeneous system of linear equations:

$$(C - \lambda_i I)v_i = 0$$

PCA Algorithm: Steps 5–6

5. Select the Top K Eigenvectors

Form a projection matrix W :

$$W = [v_1, v_2, \dots, v_K]$$

K is the number of principal components you choose to keep.

6. Project the Data onto the New Subspace

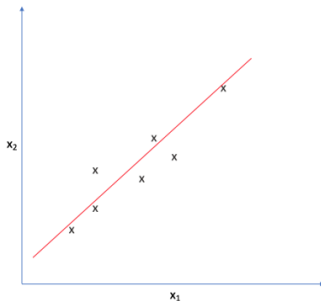
Transform the data into the new K -dimensional subspace:

$$Z = X_{\text{centered}} W$$

Z is the reduced representation of the original data.

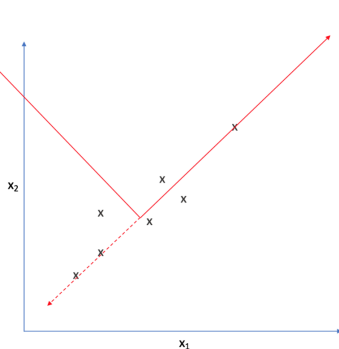
PCA Algorithm Summary

1. Standardize the data
2. Compute the covariance matrix
3. Calculate eigenvalues and eigenvectors
4. Sort eigenvectors by eigenvalues
5. Select top K eigenvectors
6. Project data onto new subspace



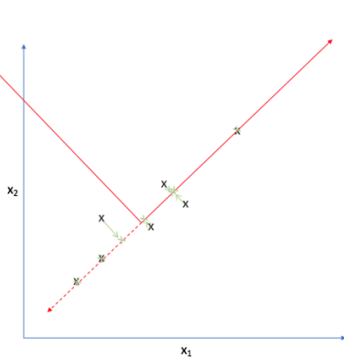
Principal Components Analysis (PCA)

Intuition: Let's look at the following two-dimensional feature-space.



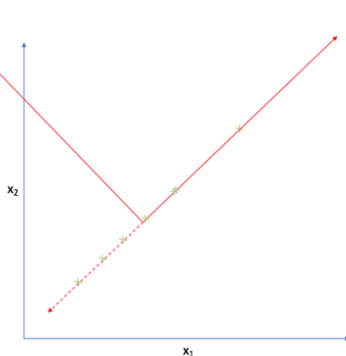
Principal Components Analysis (PCA) (cont.)

It appears that most of the points on this scatterplot lie along the following line. This suggests some degree of correlation between x_1 and x_2 .



Principal Components Analysis (PCA) (cont.)

As such, we could reorient the axes to be centered on the data and parallel to the line above.



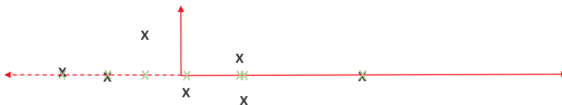
Principal Components Analysis (PCA) (cont.)

Strictly speaking, we're still dealing with two dimensions. However, let's project each observation onto the primary axis.



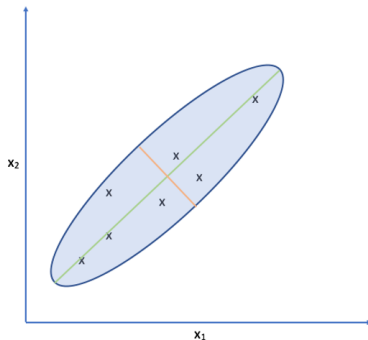
Principal Components Analysis (PCA) (cont.)

Now, every observation lies on the primary axis.



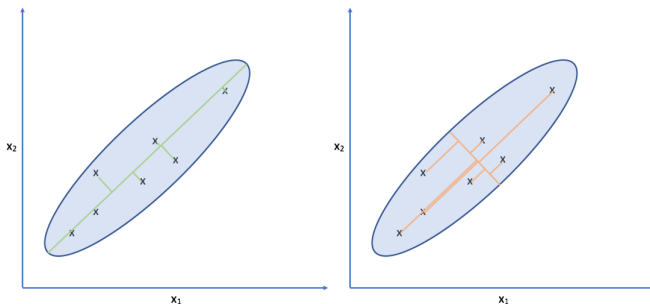
Principal Components Analysis (PCA) (cont.)

We just compressed a two dimensional dataset into one dimension by translating and rotating our axes! After this transformation, we only really have one relevant dimension and thus we can discard the second axis.



Principal Components Analysis (PCA) (cont.)

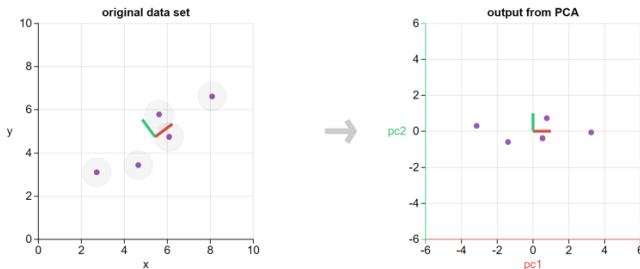
Comparing the original observations with our new projections, we can see that it's not an exact representation of our data. However, one could argue it does capture the essence of our data — not exact, but enough to be meaningful.



Principal Components Analysis (PCA) (cont.)

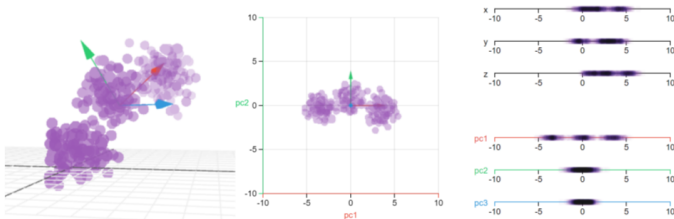
We can view the cumulative distance between our observations and projected points as a measure of information loss. Thus we'd like to orient the axes in a manner which minimizes this. How do we do this? That's where variance comes into play.

Another example: look at the spread of the data along the green and orange directions. Notice that there's a much more deviation along the green direction than there is along the orange direction.



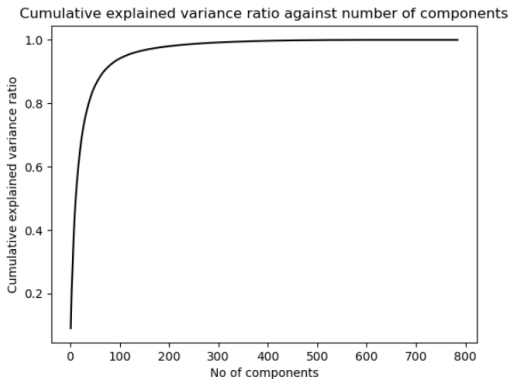
Principal Components Analysis (PCA) (cont.)

Projecting our observations onto the orange vector requires us to move much further than we would need to for projecting onto the green vector. It turns out, the vector which is capable of capturing the maximum variance of the data minimizes the distance required to move our observations as projection onto the vector.



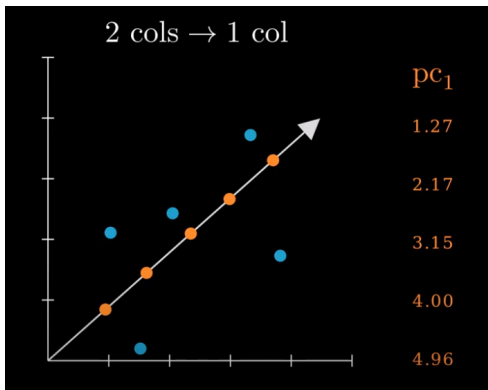
Principal Components Analysis (PCA) (cont.)

2D example



Principal Components Analysis (PCA) (cont.)

3D example



Explained Variance Ratio

Definition

The proportion of variance explained by each principal component. It determines how much information is retained in each principal component and helps in choosing the optimal number of components.

How to Choose:

- Plot the cumulative explained variance ratio.
- Select the number of components that explain a significant portion of the variance (e.g., 95%).

PCA Visualization

Video: PCA Visualization

<https://www.youtube.com/watch?v=nEvKduLXFvk>

Play: PCA 3D

https://bryanhanson.github.io/LearnPCA/articles/Vig_05_Visualizing_PCA_3D.html

Limitations of PCA

- **Linearity Assumption:** PCA assumes linear relationships between variables.
- **Sensitivity to Scaling:** Requires standardized data.
- **Loss of Information:** May discard important variance in lower components.